Cross-OEM Diagnostic Scan Tool Data Architecture Report

Vehicle identification, protocol discovery, CAN/ECU addressing, public datasets, and engineering implementation model

Prepared for diagnostic scan tool software engineering - 24 April 2026

Core conclusion: A general-purpose scan tool needs two separate data layers. The public layer can identify the vehicle and perform standards-based emissions/OBD communication. The proprietary layer is required for full OEM module coverage, module topology, non-emissions UDS DIDs, actuator tests, coding, adaptations, programming, security access, gateway routing and per-platform ECU addresses. Public data is enough to build a strong framework and generic mode; it is not enough to clone dealer tools across all OEMs.

Executive summary

This report defines the data a general-purpose diagnostic scan tool needs in order to connect to as many OEM vehicles as possible, identify the vehicle, select the relevant diagnostic transport, discover or address ECUs, and decide which services can safely be exposed to the user.

The engineering problem is not just "what CAN ID do I send?". A production scan tool needs a vehicle identity layer, protocol-detection layer, VCI abstraction, communication-parameter database, ECU/module profile database, function capability database, and a confidence model that separates verified facts from guessed or inferred behaviour.

For standardized emissions OBD, data and behaviour are well constrained by SAE/ISO standards such as J1962, J1979, ISO 15765-4, ISO 14230-4 and related WWH-OBD/OBDonUDS work [R7, R8, R11, R13]. For modern enhanced diagnostics, UDS is standardized at the service layer, but the actual DIDs, routines, security methods, coding bytes, ECU variants and gateway routing rules are OEM/platform-specific [R12]. ODX exists precisely because this data is too detailed to hard-code manually at scale; ODX describes diagnostic data, DTCs, data parameters, identification data, I/O, ECU configuration and communication parameters [R15, R16].

What the scan tool must know

- **Vehicle identity:** VIN, WMI, model year, make, model, region/market, engine, fuel type, transmission, body/platform, emissions family where available.
- **Physical interface:** J1962 pinout, power/ground availability, CAN/K-Line/J1850/DoIP availability, voltage class and VCI capabilities.
- **Protocol candidates:** OBD-II over CAN, KWP2000, ISO 9141, J1850 PWM/VPW, UDS over CAN, UDS over DoIP, J1939 for heavy-duty and off-highway cases.
- **Addressing and topology:** Functional request IDs, physical request/response IDs, ECU logical addresses, source/target addresses, gateway routing rules, functional groups, and per-network bus/channel metadata.
- **ECU definitions:** Module name, acronym, bus, request ID, response ID, protocol, diagnostic class, supported services, session/security requirements, DTC format, data identifiers, PIDs, routines and I/O controls.
- **Capability map:** Read/clear DTC, live data, freeze frame, readiness, service resets, actuator tests, adaptations, coding, programming, immobilizer/security functions, and feature availability by vehicle/platform/module.
- **Safety model:** Voltage checks, session state, tester-present cadence, gateway timeout handling, routine preconditions, write/erase protection, VIN/module mismatch handling and rollback strategy.

Public and free data sources

The following sources can legally seed a scan-tool data layer. None of them provide complete proprietary enhanced-diagnostics coverage, but they are valuable for identification, vehicle selector construction, market coverage and generic OBD behaviour.

Source	Free data available	How to use in a scan tool	Limitation
NHTSA vPIC API / downloads [R1,R2]	VIN decode, WMI, manufacturer, make/model/year/body/plant/specification fields where encoded.	Primary VIN decode and manufacturer-normalization backbone. Store API result and raw variables per VIN.	US/regulatory data bias; does not provide OEM enhanced diagnostic addresses, DIDs or routines.
NHTSA VIN Decoder [R3]	Public online decoder and VIN structure guidance.	Validation reference for VIN parsing and WMI/model-year handling.	Not a diagnostic database.
EPA FuelEconomy.gov [R4]	Downloadable vehicle data by model year: make/model, engine displacement, cylinders, fuel, drive, transmission, fuel economy fields.	Engine/fuel/transmission enrichment after VIN/year/make/model normalization.	US-market light-duty focus; not ECU/topology data.
EPA Test Car List [R5]	Downloadable test vehicle datasets in CSV/XLSX by year.	Emissions/testing metadata and variant research.	Certification/test vehicle data, not scan-tool protocol maps.
Australian Road Vehicles dataset [R6]	Registered vehicle counts by type, year, motive power, make/model, postcode/state in CSV/API formats.	Australia-specific market coverage priority and make/model normalization.	Aggregated fleet statistics, not VIN decoding or diagnostics.

SAE/ISO standards pages [R7-R15,R18]	Abstracts and purchasable standards describing the standardized behaviour.	Define legal protocol support, transport implementation and minimum generic OBD functions.	Full standards are usually paid; OEM-specific data is not included.
ASAM MCD/ODX standards [R16,R17]	ODX/MCD concepts and in some cases schemas/specification material.	Adopt ODX-inspired data model even if using your own DB first.	Actual OEM ODX packages are generally licensed/confidential.

Practical vehicle identity model

The identity layer should not treat "year/make/model" as enough. The scan tool should resolve identity in layers and carry source confidence.

Identity field	Source candidates	Why it matters
VIN	Manual entry, Mode 09 PID 02, UDS DID F190, DoIP vehicle announcement, registration/VIN scan.	Primary key for decoding and for safe module-vehicle matching.
WMI / manufacturer	VIN positions 1-3 via vPIC or local WMI table.	Normalizes OEM group and brand ownership changes.
Model year	VIN 10th character for modern VINs; vPIC decode; user override.	Selects protocol candidates and regulatory expectations.
Make / model / trim	vPIC, EPA data, AU datasets, user selection.	UI selection, coverage matching, and dataset joins.
Engine / fuel / transmission	EPA datasets, VIN decode variables, OEM/service data, user selection.	Affects ECU family, emission strategy, PID support, module list and service functions.
Market / region	VIN WMI, plant, user region, plate/registration data.	Same model may have different ECUs/protocols in AU, EU, US, JP etc.
Platform / generation	Derived internal mapping: make+model+years+market+body/engine.	Most enhanced diagnostic data should be keyed to platform/generation rather than only model name.
Build date	Door jamb/manual entry/OEM data/VIN in some cases.	Mid-year changes can alter gateway, ECU supplier and addresses.
Emission standard / OBD generation	Model year, market, engine/fuel, EPA data, OEM docs.	Determines whether to try legacy OBD, CAN OBD, WWH-OBD/OBDonUDS, DoIP etc.

Recommended normalized database schema

A useful schema separates static vehicle facts from diagnostic communication facts. Do not bury CAN IDs inside make/model rows. Addressing changes by bus, ECU, variant, protocol and model-year split.

Table	Important columns
vehicle_make	id, canonical_name, aliases, manufacturer_group, country, source_refs
vehicle_model	id, make_id, canonical_model, aliases, body_family, market_regions
vehicle_platform	id, make_id, model_ids, platform_code, generation, year_from, year_to, markets, notes
vehicle_variant	id, platform_id, engine_code, displacement, cylinders, fuel, hybrid_type, transmission, drivetrain, body, emission_standard, source_confidence
vin_decode_cache	vin_hash, decoded_year, make, model, trim, body, plant, vpic_json, decoded_at, confidence
protocol_profile	id, name, physical_layer, data_link, transport, app_layer, baudrate, address_width, connector_pins, init_sequence
vehicle_protocol_candidate	variant/platform id, protocol_profile_id, priority, evidence, first_year, last_year, confirmed_bool
network_bus	id, vehicle/platform id, bus_name, bus_type, pins, speed, addressing, gateway_required, notes
ecu_module	id, canonical_name, acronym, functional_domain, aliases
ecu_instance	id, platform/variant id, ecu_module_id, bus_id, supplier, hardware_family, software_family, year_range, optional_bool
ecu_address_profile	ecu_instance_id, request_id, response_id, functional_id, source_address, target_address, address_format, tester_source, physical_or_functional, padding, timing_profile
diagnostic_service_capability	ecu_instance_id, protocol, service, subfunction, did_pid_rid, name, data_codec_id, session_required, security_required, preconditions, read_write

data_codec	id, raw_length, endian, scale, offset, unit, enum_table, bitfields, validity_range
dtc_profile	ecu_instance_id, dtc_format, status_mask_behavior, extended_data_map, snapshot_map
routine_profile	ecu_instance_id, routine_id, name, control_type, preconditions, estimated_time, dangerous_bool
security_profile	ecu_instance_id, levels, seed_key_algorithm_ref, allowed_functions, source, legal_restriction
coverage_evidence	object_type, object_id, source, capture_id, vehicle_vin_hash, date_verified, confidence, tester_version

Protocol and transport layers

A scan tool should have a transport abstraction that separates the VCI API, bus configuration, transport segmentation, application-layer service, and decoder. This makes it possible to support ELM-style adapters for generic OBD while using J2534 or D-PDU API for serious multi-protocol work.

Layer	Examples	Implementation notes
Physical connector	SAE J1962 OBD connector, heavy-duty 9-pin Deutsch, Ethernet/DoIP connector paths.	Detect battery, grounds, CAN pair, K-Line, optional OEM pins. Protect adapter from nonstandard pin use.
VCI interface	Raw CAN API, J2534, D-PDU API, vendor SDK, ELM327-compatible subset.	J2534 and D-PDU are preferred for professional tooling; ELM clones are too limited for enhanced diagnostics.
Data link	CAN 11-bit, CAN 29-bit, CAN FD, K-Line, J1850, Ethernet.	Manage bitrate, filters, termination assumptions, receive timestamps, error frames where supported.
Transport	ISO-TP/DoCAN, KWP framing, J1939 transport protocol, DoIP TCP/UDP.	Implement segmentation, flow-control, STmin, block size, timeout, padding, sequence counters.
Application service	J1979 OBD modes, UDS services, KWP services, J1939 DM messages.	Service logic should not care whether the payload came via CAN, K-Line or DoIP.
Decoder	PID/DID/routine/DTC decoding, scaling, units, bitfields, enum interpretation.	Use data-driven codecs; never hard-code decoders into UI screens.

Standardized OBD/emissions protocol baseline

The generic mode of the tool should first support legislated emissions diagnostics. That includes connector assumptions, protocol detection, DTC read/clear, readiness, freeze-frame, VIN, calibration IDs and supported PID discovery. SAE J1979 covers OBD diagnostic test modes, SAE J1962 covers the OBD connector, and ISO 15765-4 covers CAN-based OBD/WWH-OBD communication requirements [R7,R8,R11].

Protocol family	Typical era/use	What to implement	Notes
ISO 15765-4 OBD over CAN	Most 2008+ light-duty OBD markets; often earlier.	CAN 11-bit 500k/250k and 29-bit profiles, ISO-TP, functional request, physical follow-up.	Primary modern generic OBD path.
SAE J1979 / ISO 15031 services	Generic emissions OBD.	Mode/service 01 current data, 02 freeze frame, 03 DTCs, 04 clear, 06 monitor tests, 07 pending DTC, 09 vehicle info, 0A permanent DTC where supported.	Support is capability-bit based; not every vehicle supports every PID/mode.
OBDonUDS / SAE J1979-2	Newer OBD over UDS ecosystems.	UDS-style services for emissions diagnostics where vehicle is certified to this behaviour.	Growing relevance in newer platforms.
ISO 14230-4 KWP2000	Legacy K-Line OBD, late 1990s-2000s.	5-baud/fast init as required, KWP message framing, timing and checksum handling.	Still needed for older vehicles.
ISO 9141-2	Older K-Line OBD.	Initialization and OBD services over ISO 9141 physical/data link.	Often shares J1962 pin 7 K-line.
SAE J1850 PWM/VPW	Older Ford/GM/Chrysler US OBD vehicles.	PWM and VPW physical support if your VCI supports it.	Do not assume cheap USB-CAN interfaces can do this.

CAN addressing and ECU RX/TX data

The report cannot honestly provide a universal cross-OEM table of every ECU RX/TX address, because such a table is not public and stable across all vehicles. Correct design is to store per-platform ECU addressing when known and use standards-based discovery/fallback where legally standardized.

Addressing concept	Example / pattern	Meaning
Functional request	OBD over CAN commonly uses a functional/broadcast request to emissions ECUs.	Ask any eligible ECU to answer. Good for initial generic discovery; not good for multi-frame follow-up.
Physical request	Tester sends to one ECU-specific request ID/logical target.	Required for reliable multi-frame reads, UDS sessions, routines, programming and security.
Response ID	ECU-specific response ID, often related to request ID in simple OBD mappings.	Identifies which ECU answered and should become the active physical target.
11-bit CAN ID	Base frame format.	Common for light-duty OBD and many UDS/DoCAN implementations.
29-bit CAN ID	Extended frame format with source/target addressing fields in some diagnostic profiles.	Common in heavy-duty, WWH-OBD, some OEM networks and J1939.
UDS logical address	Target/source addresses mapped into CAN, DoIP or other transport.	Data model should store logical address separately from transport-specific CAN ID.
Gateway route	Central gateway maps external diagnostic link to internal buses.	Modern vehicles may require routing activation, session setup or gateway addressing before module access.

For engineering purposes, store request/response addressing as data, not code. Your address object should include bus, protocol, address width, ID format, tester source address, ECU target address, padding byte, physical/functional flag, flow-control options, timing profile and evidence source.

Common module domains and naming normalization

Module names vary wildly by OEM. A scan tool should normalize module acronyms to canonical domains while preserving OEM labels. For example, ECM/PCM/DME/ECU may all refer to engine/powertrain control depending on brand.

Canonical domain	Common acronyms / aliases	Typical generic capability
Engine / powertrain	ECM, ECU, PCM, DME, EDC, EMS	Generic OBD, DTCs, live data, readiness, VIN/cal IDs, some actuator/routine support.
Transmission	TCM, EGS, TCU	DTCs, live data, adaptation resets, clutch/shift learn routines.
ABS / stability	ABS, ESC, ESP, DSC, VSA	DTCs, wheel speeds, steering/yaw sensors, bleed routines, sensor calibration.
Airbag / restraints	SRS, RCM, ACM, ORC	DTCs, crash status, resistance values; high caution on clears and coding.
Body control	BCM, BSI, GEM, SAM, CEM	DTCs, configuration, body I/O, lighting/locks, coding.
Gateway	CGW, GWM, ZGW, CAN Gateway	Module scan routing, network installation list, security and diagnostics firewall behaviour.
Instrument cluster	IPC, KOMBI, IC	DTCs, odometer/display data, coding; legal caution for odometer data.
HV / EV powertrain	BMS, EVC, OBC, DC/DC, inverter	DTCs, battery data, HV interlock, isolation values; safety-critical.
ADAS	ACC, AEB, Camera, Radar, Lidar, SAS	DTCs, calibration statuses, alignment routines; high precondition requirements.
Immobilizer / security	IMMO, SKREEM, PATS, CAS, KESSY, BCM security	Restricted workflows; requires legal access and audit trail.

Discovery sequence for connecting to an unknown vehicle

The following flow is the practical connection strategy for a general-purpose tool. It avoids guessing destructive services and starts with passive detection and generic OBD.

1. Electrical preflight: verify battery voltage, grounds, ignition state and adapter support. Detect whether J1962 pins indicate CAN on 6/14, K-Line on 7/15, J1850 pins, or Ethernet/DoIP path.
2. Passive listen where supported: sniff bus traffic for 1-3 seconds to infer CAN bitrate, 11/29-bit traffic, J1939 PGNs, UDS responses, gateway announcements or DoIP vehicle identification messages.
3. Try legislated OBD protocol candidates in priority order based on model year, region and observed pins. For CAN OBD, attempt ISO 15765-4 profiles first on modern vehicles.
4. Request generic supported-PID/service map. Use OBD supported-PID bitmaps before asking for specific PIDs. This prevents false assumptions and speeds scanning.
5. Read vehicle identity. Prefer VIN via generic OBD Mode 09 PID 02 where available; on enhanced UDS profiles, try DID F190 only after a valid UDS target exists.
6. Decode VIN and enrich identity using vPIC/local cache, then join to EPA/AU/open datasets for engine/fuel/model metadata where useful [R1-R6].
7. Enumerate generic responding ECUs. Record response IDs and capabilities; create a temporary session map from observed responders.
8. Select platform profile. Match VIN/year/make/model/engine/market to your internal platform coverage database. Keep confidence score and show uncertainty to the user.
9. If enhanced profile exists, scan known ECU addresses for that platform using safe read-only services first: tester present, default-session identification, DTC read, ECU identification.
10. Never run routines, actuator tests, coding, programming, security access or clears until the ECU, variant and preconditions are verified.

Protocol detection matrix

Observation	Likely path	Implementation detail
Pins 6/14 active and modern model year	CAN OBD / UDS over CAN	Try 500k 11-bit OBD first, then 250k/29-bit candidates according to region/vehicle class.
Pin 7 K-Line active, pre-CAN era vehicle	ISO 9141-2 or ISO 14230-4 KWP2000	Support init timing accurately; many low-cost adapters are unreliable here.
Heavy-duty diesel / 9-pin Deutsch / 29-bit CAN traffic	SAE J1939	Use PGN/SPN/DM model, not light-duty PID assumptions.
Ethernet/DoIP supported	UDS over DoIP	Implement ISO 13400 discovery, routing activation, TCP diagnostic channel and gateway handling [R14].
Multiple CAN networks behind gateway	Gateway-routed UDS	External DLC may only expose a gateway. Module address map must include route/gateway metadata.
No generic OBD response	Wrong ignition state, unsupported adapter, gateway locked, non-OBD machine, pins differ, or protocol mismatch.	Fall back to passive sniff and vehicle-specific profile; do not spam arbitrary IDs.

Enhanced diagnostics: UDS/KWP data required

UDS standardizes service behaviour, not the entire meaning of every byte for every OEM. ISO 14229 specifies data-link independent diagnostic services for a client tester controlling ECU diagnostic functions [R12]. The missing OEM-specific data is exactly what ODX/MCD data is designed to carry [R15,R16].

UDS/KWP item	Data needed in your DB	Why it matters
Session control	Supported sessions, session IDs, P2/P2* timing, tester-present cadence.	Many functions require extended/programming sessions; wrong timing drops session.
ECU reset	Allowed reset types, preconditions, post-reset delay.	Can disrupt vehicle networks if used carelessly.
Read data	DID list, names, codecs, scaling, units, sessions, security requirements.	Live data/enhanced data depends on this.
Write data	Writable DIDs, codecs, range checks, dependencies, backup/restore strategy.	High risk; requires exact variant match.
DTC services	DTC format, status mask, groups, snapshot/extended data IDs and decoders.	Needed for meaningful enhanced fault reports.
Routine control	RID list, start/stop/result parameters, preconditions, timing, danger classification.	Service functions, calibrations, regens, bleeding and relearns are usually routines.

Input/output control	Controllable DIDs/IO channels, control parameters, exit strategy.	Actuator tests must release control safely.
Security access	Security levels, seed/key method reference, allowed operations, audit/legal restrictions.	Cannot be generic; must be lawfully sourced and protected.
Request download/upload/transfer	Memory segments, compression/encryption method, block size, checksum, programming sequence.	Firmware programming requires exact flash data and robust recovery.
Communication control	Which comms can be disabled, recovery method.	Can brick communication temporarily if mishandled.

ODX-inspired runtime architecture

Even without licensed OEM ODX packages, use an ODX-like model internally. ODX is explicitly intended to describe diagnostic trouble codes, data parameters, identification data, input/output parameters, ECU configuration/variant coding and communication parameters [R15]. ASAM states that an ODX file can configure compliant testers/D-servers for ECU communication without reprogramming the tester for every ECU [R16].

Runtime component	Responsibility
Vehicle resolver	Turn VIN/manual selection into platform/variant candidates with confidence.
Protocol manager	Own VCI channels, bitrate, filters, transport protocol and timing profiles.
Diagnostic kernel	Execute services by symbolic name: ReadVIN, ReadDTCs, ReadDID, StartRoutine, etc.
Codec engine	Convert raw bytes to physical values and encode write/routine parameters.
Capability resolver	Decide which functions are visible for vehicle/module/session/security state.
Safety/precondition engine	Block actions if voltage, ignition, engine state, gear, speed, HV state or module identity is unsafe/unknown.
Evidence logger	Capture raw request/response, vehicle identity, module identity, software version, timestamp and tool version.
Update system	Ship database updates separately from executable updates; allow coverage fixes without recompiling.

J2534, D-PDU API and adapter strategy

A cross-manufacturer scan tool should not be hard-coupled to one adapter. SAE J2534 exists as a pass-thru interface for vehicle programming; the SAE page describes its purpose as enabling reprogramming of emission-related control modules in 2004 and later model year vehicles [R9]. Bosch notes the US EPA mandate for OEM J2534 support from model year 2004+ powertrain ECUs, with provisions around 1996-2003 support [R10]. ISO 22900-2 defines the D-PDU API as the MVCI protocol module interface and common basis for diagnostic/reprogramming applications [R18].

Adapter class	Pros	Cons	Recommended use
ELM327/STN-style serial adapter	Cheap, easy generic OBD, AT command simplicity.	Limited timing, filtering, multi-protocol and enhanced diagnostic support; clone quality varies.	Basic OBD reader mode only.
USB-CAN adapter	Excellent raw CAN control and logging.	Usually no J1850/K-Line/J2534; no OEM programming compatibility by itself.	Engineering, reverse-validation, CAN/UDS research on supported pins.
J2534 pass-thru	Professional route for OEM reprogramming and many diagnostics; common Windows API model.	Vendor DLL differences; not all protocols/features available on every device.	Primary professional Windows VCI abstraction.
D-PDU API / MVCI	Standardized VCI access abstraction, strong fit for ODX/MCD architecture.	Less common in hobby ecosystem; commercial stack complexity.	High-end diagnostic platform architecture.
DoIP/Ethernet interface	High throughput, modern vehicle support.	Requires ISO 13400 routing/discovery/security handling.	Modern OEM platforms, programming, diagnostics over Ethernet.

Coverage data lifecycle

Cross-OEM capability is built, not found in one free database. Your database must track provenance and confidence, otherwise it becomes a pile of dangerous guesses.

Confidence level	Meaning	UI / execution policy
0 - Unknown	No data or only user guess.	Generic OBD only. Do not expose enhanced actions.
1 - Inferred	Inferred from platform/year or similar vehicle.	Read-only probing allowed with user-visible uncertainty.
2 - Observed	Seen on a live vehicle/log but not fully validated.	Read-only functions okay; write/routine blocked unless specifically validated.
3 - Verified	Tested on exact platform/variant/module/software range.	Expose relevant functions with normal warnings.
4 - OEM/licensed	Sourced from OEM/ODX/service data with versioning.	Preferred data path; still validate ECU identity before dangerous functions.

Minimum captured evidence per vehicle session

- VIN or VIN unavailable reason; decoded WMI/year/make/model/engine/market if known.
- VCI model, firmware, API, bus/channel, bitrate, ID format and protocol.
- Raw request/response log with timestamps and transport-level errors.
- All responding ECU addresses and module identification responses.
- DTCs with raw status bytes, snapshots and extended data where read.
- Service support maps, DID/PID support bitmaps and negative response codes.
- Ignition/battery/engine state and preconditions for any routine or write function.
- Database version and profile used, including confidence/evidence IDs.

Implementation blueprint for cross-manufacturer scanning

The most robust design is a staged scan engine with strict separation between generic discovery and vehicle-specific enhanced capability.

Stage	Purpose	Inputs	Outputs
1. Connect	Establish electrical and VCI connection.	Adapter, connector pins, voltage, ignition.	Available channels and candidate protocols.
2. Generic probe	Find standardized OBD path.	Protocol candidates from model year/pins/sniffing.	Responding generic ECUs, OBD capability map, VIN if available.
3. Identify	Resolve vehicle identity.	VIN/vPIC/EPA/AU/open data/manual input.	VehicleVariant candidates and confidence.
4. Match profile	Choose best internal platform profile.	Vehicle identity + observed bus/ECU evidence.	Protocol profiles, ECU address candidates.
5. Enhanced read-only scan	Read module list, IDs and DTCs.	Known safe ECU addresses and services.	Module report, actual installed ECUs, verified identities.
6. Capability enablement	Expose safe functions only.	Module identity, session state, database confidence.	UI function list with warnings/preconditions.
7. Service execution	Run routines/writes/programming with guardrails.	Exact profile, precondition pass, backup data.	Audited operation result and recovery info.
8. Learn/update	Improve coverage database.	Logs, user validation, source evidence.	New evidence records and potential profile updates.

Recommended API/service abstractions

Design symbolic APIs first. A UI should call ReadDTCs(module), not send raw CAN frames. Raw frames belong in profile data and transport drivers.

API surface	Example methods
VehicleIdentityService	DecodeVin(), ResolveVehicle(), MatchPlatform(), GetVehicleCandidates()

ProtocolDiscoveryService	DetectPins(), PassiveSniff(), ProbeObdProtocols(), ProbeDoip(), ProbeJ1939()
DiagnosticSession	OpenEcu(), StartTesterPresent(), ChangeSession(), KeepAlive(), Close()
GenericObdService	ReadSupportedPids(), ReadCurrentData(), ReadFreezeFrame(), ReadDtc(), ClearDtc(), ReadVehicleInfo()
UdsService	ReadDID(), WriteDID(), ReadDTCInfo(), RoutineControl(), IOControl(), SecurityAccess(), RequestDownload()
ModuleScanner	ScanKnownAddresses(), IdentifyModule(), ReadModuleDtc(), ReadModuleLiveData()
SafetyGuard	CheckVoltage(), CheckIgnition(), CheckPreconditions(), RequireExactModuleMatch(), RequireBackup()
EvidenceStore	SaveSessionLog(), SaveRawFrame(), SaveProfileMatch(), SaveCoverageObservation()

What cannot be solved by free public datasets

The following data is not realistically available as a complete, legal, free public database across all OEMs. Your architecture should assume these fields require OEM subscriptions, licensed data, ODX packages, standards purchases, field validation, or legally obtained reverse-engineering/engineering logs.

- Full per-OEM ECU RX/TX addressing for every module, year, platform, market and gateway variant.
- Complete UDS DID lists and byte-level scaling for all modules.
- Actuator tests and routine IDs with correct parameters/preconditions.
- Coding/adaptation datasets and variant configuration bit meanings.
- Seed-key algorithms, security levels and immobilizer/key programming workflows.
- Firmware memory maps, erase/programming sequences, flash containers and checksums.
- Gateway unlock/routing rules and authenticated diagnostics for modern secure gateways.
- Safety-critical ADAS/HV calibration data and legal authorization workflows.

Data ingestion plan

Build an ingestion pipeline that allows you to mix public datasets, manual engineering data and future licensed/OEM data without rewriting the scan tool.

11. Create raw_import tables for every source. Never throw away raw source rows; normalize into canonical tables separately.
12. Import vPIC manufacturer/WMI/VIN decode data and cache decode results by VIN hash, not by storing customer VINs unnecessarily.
13. Import EPA vehicles.csv and test-car lists for year/make/model/engine/fuel/transmission enrichment.
14. Import Australian road-vehicle datasets for market coverage prioritization and make/model alias cleanup.
15. Create canonical make/model/platform alias tables. Expect spelling differences, brand group changes and market-specific names.
16. Define protocol profiles for standardized protocols before adding any OEM-enhanced data.
17. Add ECU/module profiles incrementally with evidence and confidence. Do not silently merge guessed addresses into verified data.
18. Use migration/versioning for database changes. Diagnostic data should have semantic versioning and rollback.

Starter data objects - examples

Object	Example fields
ProtocolProfile: ISO15765_OBD_CAN_11_500	bus=CAN, id_width=11, bitrate=500000, transport=ISO-TP, functional_request=standard OBD functional ID, padding=0x00/0xCC configurable
ProtocolProfile: ISO14230_KWP_FAST_INIT	physical=K-Line, init=fast-init, baud=10400, app=KWP2000, timing=P1/P2/P3/P4 profile
ProtocolProfile: UDS_DoIP	physical=Ethernet, discovery=UDP, diagnostic_channel=TCP, routing_activation=true, app=UDS

EcuModule: EngineControl	canonical=Engine Control Module, aliases=ECM/PCM/DME/EDC/EMS, domain=powertrain
EcuAddressProfile	platform_id, ecu=EngineControl, bus=PowertrainCAN, request_id, response_id, functional_id, id_format, evidence_id
Capability: ReadVIN	protocol=OBD Mode 09 PID 02 or UDS DID F190, preconditions=ignition_on, safe=true, read_only=true

Safety and legal engineering notes

A scan tool that can only read OBD PIDs is low risk. A scan tool that can run routines, write configuration, erase memory or access security functions is high risk. Treat the enhanced layer like industrial control software, not a toy.

- Never infer write/routine/programming capability from a similar model without exact module identity confirmation.
- Before clear-DTC, warn the user that freeze-frame and readiness data may be lost and emissions readiness may reset.
- Before actuator tests, verify engine state, vehicle speed, gear, parking brake, HV state and user confirmation as appropriate.
- Before coding/adaptation, read and store original values, module identity, session log and battery voltage.
- Before programming, require stable power supply detection and verify flash package compatibility using OEM rules.
- For security/immobilizer functions, use lawful OEM/authorized access paths, audit logs and regional compliance workflows.

Suggested development roadmap

Phase	Build	Definition of done
1. Generic OBD core	J1962/J2534 or adapter abstraction, ISO-TP, J1979 modes, VIN decode, DTCs, live data.	Works on common CAN OBD vehicles and handles no-response/timeouts correctly.
2. Vehicle identity DB	vPIC + EPA + AU datasets + alias/canonicalization.	VIN/manual selection resolves to stable internal VehicleVariant candidates.
3. Scan engine	Protocol discovery, generic ECU response map, logging, evidence store.	Produces repeatable scan reports with raw evidence.
4. Enhanced UDS framework	UDS service client, sessions, DIDs, DTC status, negative response handling.	Can perform read-only UDS against known safe test ECUs/profiles.
5. Profile database	ODX-inspired schema for modules, addressing, DIDs, routines, codecs and confidence.	New vehicle coverage can be added without recompiling code.
6. Gateway/DoIP/J1939	DoIP discovery/routing, J1939 PGN/SPN/DM support, gateway route model.	Supports modern Ethernet diagnostics and heavy-duty read-only cases.
7. Service functions	Routines, actuator tests, adaptations with precondition engine.	Only verified functions exposed; logs and backups created automatically.
8. Programming/security	J2534/OEM integration, lawful security workflows, flash safety systems.	Use OEM data/access and strict recovery/audit mechanisms.

Appendix A - Canonical scan-tool data checklist

- VIN decode and vehicle identity: VIN, WMI, model year, make, model, trim, platform, engine, fuel, transmission, region, build date.
- Connector and electrical: connector type, pins, battery/ground pins, bus pins, voltage, ignition detection method.
- Protocol candidates: standard, physical/data link, transport, app layer, bitrate, init sequence, timings, addressing mode.
- Network topology: buses, gateway, exposed diagnostic path, module-to-bus mapping, routing requirements.
- ECU address data: module, request ID, response ID, functional ID, source/target address, tester address, 11/29-bit flag, padding, flow-control parameters.
- ECU metadata: module name/acronym/domain, supplier, hardware/software IDs, part numbers, calibration IDs, VIN write/read availability.

- Service support: OBD modes, UDS/KWP services, DIDs/PIDs/RIDs, I/O controls, routines, DTC services, supported sessions/security levels.
- Decoding: byte lengths, scale/offset/unit, endian, bitfields, enum maps, validity ranges, text translations.
- Preconditions: voltage, ignition, engine running/stopped, speed, gear, brake, temperature, HV safety state, network quiet/awake state.
- Legal/safety: security access authority, immobilizer restrictions, emissions rules, audit logs, user consent, service warnings.

Appendix B - Source references

R1. NHTSA vPIC API. Public VIN/manufacturer/vehicle specification API populated from manufacturer 565 submissions. https://vpic.nhtsa.dot.gov/api/

R2. NHTSA vPIC Downloads. Downloadable/offline vPIC database resources. https://vpic.nhtsa.dot.gov/Downloads

R3. NHTSA VIN Decoder. Public VIN decoder and plant/manufacturer identification guidance. https://www.nhtsa.gov/vin-decoder

R4. EPA FuelEconomy.gov Downloads. Downloadable model-year vehicle fuel economy files with make/model/engine/transmission metadata. https://www.fueleconomy.gov/feg/download.shtml

R5. EPA Test Car List. Downloadable EPA test vehicle data for fuel economy/emissions testing. https://www.epa.gov/compliance-and-fuel-economy-data/data-cars-used-testing-fuel-economy

R6. Road Vehicles Australia, January 2025. Australian public road-vehicle statistics, including make/model/year/motive power CSV/API resources. https://www.data.gov.au/data/dataset/road-vehicles-australia-january-2025

R7. SAE J1979. E/E diagnostic test modes / emissions OBD communication with test equipment. https://www.sae.org/standards/j1979-e-e-diagnostic-test-modes

R8. SAE J1962. Standard diagnostic connector intended to satisfy OBD connector requirements. https://www.sae.org/standards/j1962_201509-diagnostic-connector

R9. SAE J2534. Pass-thru interface for vehicle programming, especially emission-related modules. https://www.sae.org/standards/j2534-1_5_00-recommended-practice-pass-thru-vehicle-programming

R10. Bosch J2534 FAQ. Practical J2534 notes, including EPA-mandated OEM support from 2004+ powertrain ECUs. https://www.boschdiagnostics.com/j2534-faq

R11. ISO 15765-4. Requirements for CAN networks used for OBD/WWH-OBD external test equipment. https://www.iso.org/standard/67245.html

R12. ISO 14229-1 UDS. Data-link independent UDS diagnostic services allowing a client tester to control ECU diagnostic functions. https://www.iso.org/standard/72439.html

R13. ISO 14230-4 KWP2000. KWP2000 OBD requirements for vehicle and scan tool communication. https://www.iso.org/standard/28826.html

R14. ISO 13400-2 DoIP. Diagnostic communication over IP, including discovery and TCP/UDP diagnostic communication. https://www.iso.org/standard/74785.html

R15. ISO 22901-1 ODX. ODX data model for diagnostic data including DTCs, data parameters, identification, I/O, variant coding, and communication parameters. https://www.iso.org/standard/41207.html

R16. ASAM MCD-2 D / ODX. ODX files describe ECU-external tester communication and translate raw diagnostic messages to human-readable values. https://www.asam.net/standards/detail/mcd-2-d/

R17. ASAM MCD-3 D. API of a diagnostic kernel that interprets hex diagnostic messages into human-readable values. https://www.asam.net/standards/detail/mcd-3-d/

R18. ISO 22900-2 D-PDU API. MVCI D-PDU API protocol module interface for diagnostic/reprogramming applications. https://www.iso.org/standard/62490.html